

NORAIZ RANA

Task #17

Topic

React state management
with Context API

Date:

2 August 2025

SUBMITTED TO:

APPVERSE TECHNOLOGIES

Learning Report: State Management with React Context API

Styling & Components Architecture Module

1. Introduction

React provides a powerful way to build interactive user interfaces using components. However, as applications grow, managing shared state across multiple components becomes challenging—especially when passing props through deeply nested components. This is where the **Context API** comes in.

The **Context API** is a built-in feature of React that allows data to be shared globally across the component tree without explicitly passing props at every level. It is ideal for managing application-wide state such as authentication, themes, user preferences, or settings.

3. Core Concepts

Term	Purpose
<code>createContext()</code>	Creates a new context object
<code><Provider></code>	Makes context value available to children
<code>useContext()</code>	Accesses context value inside functional components

4. Context API Workflow Example

Step 1: Create Context

```
js
```

```
CopyEdit
import { createContext } from 'react';
export const ThemeContext = createContext();
```

Step 2: Create a Provider

```
js
CopyEdit
import { useState } from 'react';
import { ThemeContext } from '../ThemeContext';

export function ThemeProvider({ children }) {
  const [darkMode, setDarkMode] =
    useState(false);
  const toggleTheme = () =>
    setDarkMode(!darkMode);

  return (
    <ThemeContext.Provider value={{ darkMode,
    toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Step 3: Wrap App

```
js
CopyEdit
<ThemeProvider>
  <App />
</ThemeProvider>
```

Step 4: Consume Context

```
js
CopyEdit
```

```
const { darkMode, toggleTheme } =  
useContext(ThemeContext);
```

5. Practical Applications

- **Theme Management** – toggle dark/light modes
 - **Authentication** – store and access login state
 - **Cart Functionality** – access cart data across components
 - **User Preferences** – language, display settings, etc.
-

7. When NOT to Use Context

- If the state changes frequently (e.g., real-time search inputs), Context may cause **unnecessary re-renders**.
 - For complex or large-scale applications, consider advanced tools like **Redux**, **Zustand**, or **Jotai**.
-

8. Key Benefits

- Cleaner code with minimal prop drilling
- Easier to manage shared global state
- Better component separation and modularity
- Improved maintainability and scalability